



[Home](#) / [Manuals](#) / [Docker Compose](#) / [How-tos](#) / [Networking](#)

Networking in Compose

Table of contents

Default network and service discovery

Updating containers on the network

Change the network mode

Specify custom networks

Internal networks

Configure the default network

Use an existing external network

Connecting multiple Compose projects

Hybrid networking

Custom DNS with `extra_hosts`

Multi-host networking

Link containers

Debugging

Inspect port mappings

Verify network membership

Check connectivity

Further reference information

Compose handles networking for you by default, but gives you fine-grained control when you need it. This page explains how the default network works and how containers discover each

other by name. It also covers when and how to define custom networks, connect services across separate Compose projects, map custom hostnames, and debug connectivity issues.

Default network and service discovery

By default, Compose sets up a single [network](#) for your app. Each container for a service joins the default network and is both reachable by other containers on that network, and discoverable by its service name. This network uses the `bridge` driver. To attach services to a different Compose network or an external one, see [Specify custom networks](#). To change how a service uses the host network stack (`network_mode: host`), see [Change the network mode](#).

For most development setups, the default network is sufficient. When you run `docker compose up` , Compose creates a network named `<project-name>_default` and attaches all services to it. Each service registers its name with an internal DNS server, so containers can reach each other using the service name directly. No IP addresses or manual configuration is needed.

For example, suppose your app is in a directory called `myapp` , and your `compose.yaml` looks like this:

```
services:
  web:
    build: .
    ports:
      - "8000:8000"
  db:
    image: postgres:latest
    ports:
      - "8001:5432"
```

Compose automatically connects all services to the default network, so you don't need to define `networks` explicitly in the Compose file.

When you run `docker compose up` , the following happens:

1. A network called `myapp_default` is created.
2. A container is created using `web` 's configuration. It joins `myapp_default` under the name `web` .

3. A container is created using `db`'s configuration. It joins `myapp_default` under the name `db`.

Each container can now look up the service name `web` or `db` and get back the appropriate container's IP address. The `web` service can connect to the database at `postgres://db:5432`. From the host machine, the same database is accessible at `postgres://localhost:8001` if your container is running locally.

Tip

Docker assigns container IP addresses dynamically from the network's subnet each time a container starts so they are not persisted across restarts or recreations. This means you should always reference services by name, not IP address. When containers are recreated, for example after a configuration change, they receive a new IP address. The service name stays stable.

Your app's network is given a name based on the "project name", which is taken from the name of the directory it lives in. You can override the project name with either the `--project-name` flag or the `COMPOSE_PROJECT_NAME` environment variable.

The `HOST_PORT` and `CONTAINER_PORT` serve different purposes. In the example above, for `db`, the `HOST_PORT` is `8001` and the container port is `5432` (the Postgres default). Networked service-to-service communication uses the `CONTAINER_PORT`. The host port is only used when accessing the service from outside the network.

Updating containers on the network

If you make a configuration change to a service and run `docker compose up` to update it, the old container is removed and the new one joins the network under a different IP address but the same name. Running containers can look up that name and connect to the new address, but the old address stops working.

If any containers have connections open to the old container, they are closed. It is each container's responsibility to detect this condition, look up the name again, and reconnect.

Change the network mode

By default, each service joins the project's bridge network. It is the most secure networking mode. If you don't specify `network_mode`, this is the type of network you are creating.

You can override the networking mode on a per-service basis. The `network_mode` option accepts the following values:

- `host` : The container shares the host's network stack. No port mapping is needed or supported, and service name DNS resolution does not work. Use for system-level tools like network monitors that require direct access to host interfaces. A container using `network_mode: host` can access all host ports and observe all network traffic on the host. Use it only when genuinely required.
- `none` : Turns off all container networking.
- `service:{name}` : Gives the container access to the specified container by referring to its service name.
- `container:{name}` : Gives the container access to the specified container by referring to its container ID.

You can mix modes in a single project:

```
services:
  app:
    image: myapp
    networks:
      - isolated
    ports:
      - "3000:3000"

  monitoring:
    image: netdata/netdata
    network_mode: host # Can monitor host system and all host ports

networks:
  isolated:
    driver: bridge
```

Specify custom networks

Instead of just using the default app network, you can specify your own networks with the top-level `networks` key. This lets you create more complex topologies and specify [custom network drivers](#) and options. You can also use it to connect services to externally-created networks which aren't managed by Compose.

Each service can specify what networks to connect to with the service-level `networks` key, which is a list of names referencing entries under the top-level `networks` key.

The following example shows a Compose file which defines two custom networks. The `proxy` service is isolated from the `db` service, because they do not share a network in common. Only `app` can talk to both.

```
services:
  proxy:
    build: ./proxy
    networks:
      - frontend
  app:
    build: ./app
    networks:
      - frontend
      - backend
  db:
    image: postgres:latest
    networks:
      - backend

networks:
  frontend:
    driver: bridge # Specify driver options
    driver_opts:
      com.docker.network.bridge.host_binding_ipv4: "127.0.0.1"
  backend:
    driver: custom-driver # Use a custom driver
```

Networks can be configured with static IP addresses by setting the [ipv4_address](#) and/or [ipv6_address](#) for each attached network.

Networks can also be given a [custom name](#):

```
services:
  # ...
networks:
  frontend:
    name: custom_frontend
    driver: custom-driver-1
```

Internal networks

Setting `internal: true` on a network creates it without a connection to the host's network interfaces. It has no default gateway for external connectivity. This is useful for services like databases that should be completely unreachable from outside the container network:

```
services:
  cache:
    image: redis
    networks:
      - isolated

  worker:
    image: myworker
    networks:
      - isolated
      - public

networks:
  isolated:
    internal: true  # No external connectivity
  public:  # Standard bridge network, created by Compose on docker compose up
```

Note that a service connected to both an internal and a non-internal network (like `worker` above) can still reach the internet via the non-internal network `public`.

Configure the default network

Instead of, or as well as, specifying your own networks, you can also change the settings of the app-wide default network by defining an entry under `networks` named `default`:

```

services:
  web:
    build: .
    ports:
      - "8000:8000"
  db:
    image: postgres:latest

networks:
  default:
    driver: custom-driver-1    # Use a custom driver

```

Use an existing external network

If you've manually created a bridge network using `docker network create`, you can connect your Compose services to it by marking the network as `external`:

```

services:
  # ...
networks:
  network1:
    name: my-pre-existing-network
    external: true

```

Instead of creating `<project-name>_default`, Compose looks for a network called `my-pre-existing-network` and connects your containers to it.

Connecting multiple Compose projects

External networks are particularly useful when services in separate Compose projects need to communicate. Create a shared network once, then reference it as `external` in each project:

```
docker network create inter-project
```

backend-compose.yaml:

```
services:
```

```
api:
  image: myapi:latest
  networks:
    - shared
    - default    # Also keep the project's internal network

networks:
  shared:
    external: true
    name: inter-project
```

frontend-compose.yaml:

```
services:
  web:
    image: myfrontend:latest
    environment:
      API_URL: http://api:8080    # Reference by service name
    networks:
      - shared

networks:
  shared:
    external: true
    name: inter-project
```

Services on the same external network can reach each other by service name, just like services within a single project.

Important

The external network must exist before you run `docker compose up`. If it doesn't, Compose fails with a `Network not found` error. Always create it first with `docker network create`.

Hybrid networking

A service can belong to both an external shared network and its own project-internal network. This lets you expose only the services that need to be reachable from other projects, while keeping everything else, such as databases, fully isolated:

```
services:
  api:
    image: myapp-api
    networks:
      - shared      # Reachable from other projects
      - internal    # Can also reach the database

  database:
    image: postgres:latest
    networks:
      - internal    # Not exposed on the shared network

networks:
  shared:
    name: inter-project
    external: true
  internal: {}      # Project-specific, isolated
```

Custom DNS with `extra_hosts`

You can add custom hostname-to-IP mappings to a container's `/etc/hosts` file using `extra_hosts`. This is useful when a service needs to resolve a hostname that isn't registered in Docker's internal DNS. For example, a fixed-IP dependency or a staging endpoint:

```
services:
  app:
    image: myapp
    extra_hosts:
      - "api.staging:192.168.1.100"
      - "cache.internal:192.168.1.101"
```

To map a hostname dynamically to the host machine's IP, use the special `host-gateway` value:

```
services:
```

```
app:
  image: myapp
  extra_hosts:
    - "host.docker.internal:host-gateway"
```

On Linux, `host-gateway` resolves to the host's IP on the default bridge network. On Mac and Windows, Docker automatically provides this, `host-gateway` resolves to the same internal IP address as `host.docker.internal`.

You can also drive `extra_hosts` from environment variables, which makes it easy to point services at different targets per environment:

```
services:
  app:
    image: myapp
    extra_hosts:
      - "api.service:${API_HOST:-127.0.0.1}"
      - "auth.service:${AUTH_HOST:-127.0.0.1}"
```

Where `.env.development` might set `API_HOST=localhost` and a production env file might set `API_HOST=10.0.1.50`.

To verify what has been injected, inspect the hosts file inside the container:

```
$ docker compose exec app cat /etc/hosts
```

Multi-host networking

When deploying a Compose application on a Docker Engine with [Swarm mode enabled](#), you can use the built-in `overlay` driver to enable multi-host communication. Overlay networks are always created as `attachable`. You can optionally set the `attachable` property to `false`.

To learn more, see the [overlay network driver documentation](#).

Link containers

Links allow you to define extra aliases by which a service is reachable from another service. They are not required for basic service-to-service communication. By default, any service

can reach any other service at that service's name. In the following example, `db` is reachable from `web` at both the hostnames `db` and `database` :

```
services:
  web:
    build: .
    links:
      - "db:database"
  db:
    image: postgres:latest
```

See the [links reference](#) for more information.

Debugging

When a service can't reach another, work through the following steps in order: first confirm the network configuration looks right, then confirm the containers are actually attached, then test live connectivity.

Inspect port mappings

To find out which host port maps to a container port, use `docker compose port` :

```
# Which host port maps to container port 5432 on db?
$ docker compose port db 5432
# Output: 0.0.0.0:8001
```

This is especially useful when using dynamic port mapping, where the host port changes on every `docker compose up` :

```
services:
  web:
    image: nginx
    ports:
      - "80" # Docker assigns the host port dynamically
```

```
$ docker compose port web 80
```

```
# Output: 0.0.0.0:55432
```

When you scale a service, each replica gets its own dynamic port. Use `--index` to query a specific replica:

```
$ docker compose up -d --scale web=3

$ docker compose port --index=1 web 80 # Output: 0.0.0.0:55001
$ docker compose port --index=2 web 80 # Output: 0.0.0.0:55002
$ docker compose port --index=3 web 80 # Output: 0.0.0.0:55003
```

By default, `docker compose port` looks for TCP mappings. If a service exposes both TCP and UDP on the same port, use `--protocol` :

```
$ docker compose port --protocol=udp myservice 53
```

Verify network membership

To check which containers are attached to a network (useful when troubleshooting connectivity across external or custom networks):

```
$ docker network inspect <network-name>
```

Check connectivity

If the network membership looks correct but services still can't reach each other, test connectivity from inside a running container using `docker compose exec` .

Further reference information

For full details of the network configuration options available, see the following references:

- [Top-level networks element](#)
- [Service-level networks attribute](#)

